EE 3150: Signals and Systems

Convolution and Filtering

Colden Briggs, Eben Radocchia, and Liam Donlan
Instructor: Dr. Ossareh | University of Vermont

October 15, 2026

Contents

Introduction **2**

Part 1: Discrete-Time Convolution **3**

Part 2: Continuous-Time Convolution **7**

Task 2.1: Step Response of the First Stage 7

Task 2.2: Cosine Response of the First Stage 8

Task 2.3: The Two-Stage Cascade 9

Part 3: FIR Audio Filters **11**

Task 3.1: Audio Filtering 11

Task 3.2: Order Does Not Matter 12

Task 3.3: Moving Average Gain versus Frequency 13

Part 4: IIR Filtering **15**

Task 4.1: IIR Streaming 15

Task 4.2: Audio Filtering and FIR/IIR Comparison 17

Task 4.3: IIR Gain versus Frequency 19

Conclusion **20**

Introduction

[write here]

Part 1: Discrete-Time Convolution

Task 1: Mechanics of Convolution

The input sequences are

$$x[n] = \{1, 2, 3, 1\}, \quad n = 0, \ldots, 3, \qquad h[n] = \{1, -1, 2\}, \quad n = 0, \ldots, 2.$$

Hand calculation

Figure placeholder

p1_hand_calc.png

Figure 1: Handwritten flip-and-slide calculation of $x[n] * h[n]$.

The convolution gives

$$y[n] = \{1, 1, 3, 2, 5, 2\}, \qquad n = 0, \ldots, 5.$$

The expected output length is

$$N_y = N_x + N_h - 1 = 4 + 3 - 1 = 6.$$

The result has six samples, which matches the expected length.

Python code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 1])
h = np.array([1, -1, 2])


# x and h both start at n = 0
def conv_loop(x, h):

    Ny = len(x) + len(h)- 1
    y = np.zeros(Ny)
    for n in range(Ny): # compute each output sample
        for k in range(len(x)): # add the overlapping samples
            m = n - k
            if 0 <= m < len(h):
                y[n] = y[n] + x[k]*h[m]
    return y


y1 = conv_loop(x, h)
y2 = np.convolve(x,h)
n1 = np.arange(len(y1))
n2 = np.arange(len(y2))

fig1, ax1 = plt.subplots(figsize=(8, 6))
fig2, ax2 = plt.subplots(figsize=(8, 6))

ax1.stem(n1, y1, label="y1[n] = x[n] * h[n]")

ax1.set_title("Convolution output")
ax1.set_xlabel("n")
ax1.legend()
ax1.grid(True)
fig1.tight_layout()

ax2.stem(n2, y2, label="y2[n] = x[n] * h[n]")
ax2.set_title("Convolution output 2")
ax2.set_xlabel("n")
ax2.legend()
ax2.grid(True)
fig2.tight_layout()
plt.show()
```

Convolution results

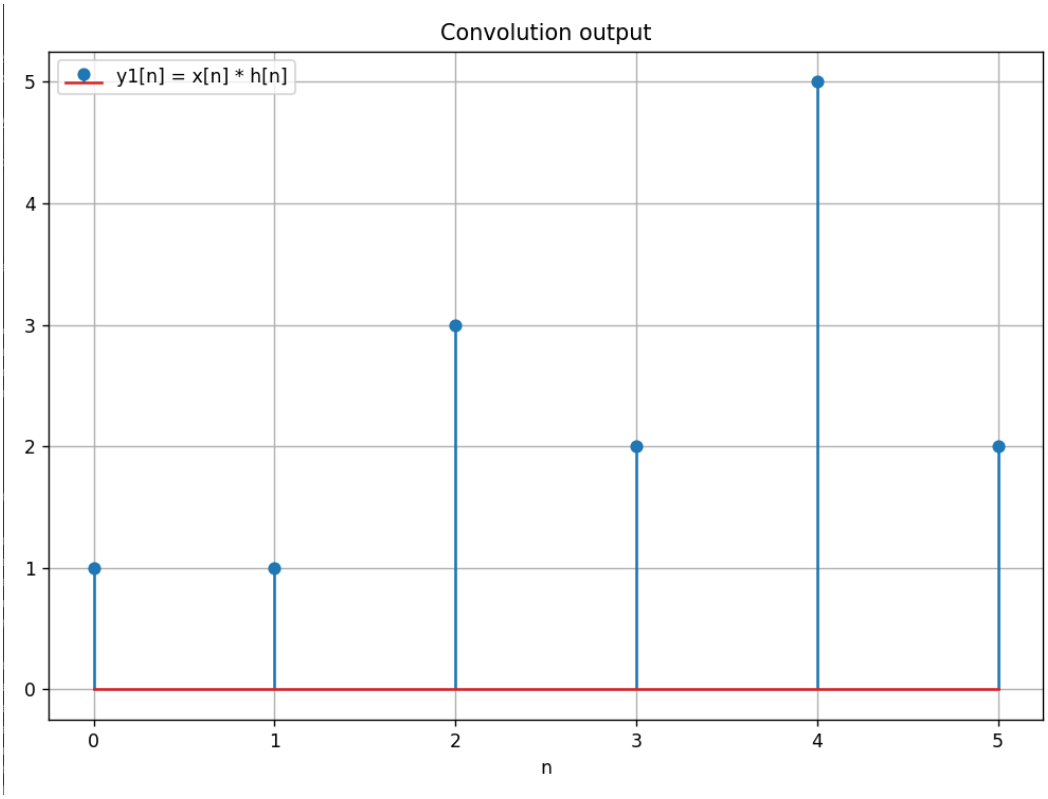


Figure 2: $y[n]$ computed with `conv_loop`.

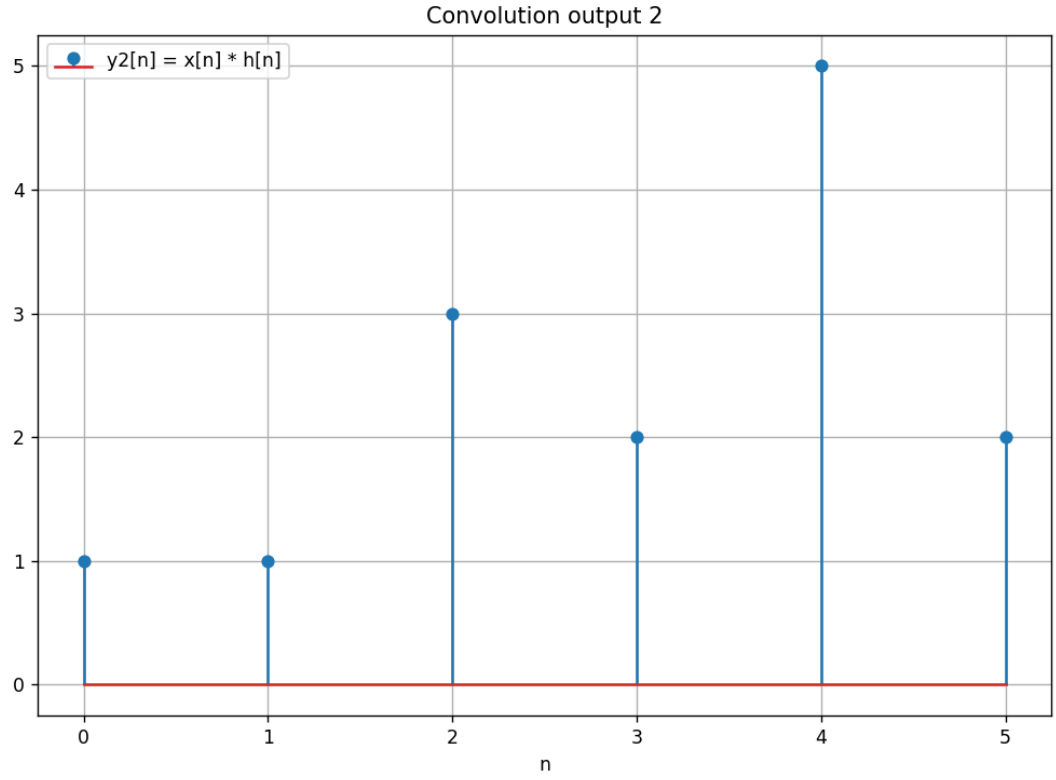


Figure 3: $y[n]$ computed with `np.convolve`.

The handwritten result, the `conv_loop` output, and the `np.convolve` output all give $y[n] = \{1, 1, 3, 2, 5, 2\}$. The two stem plots are identical, so all three methods agree.

Convolution with a shifted impulse

$$x[n] * \delta[n-2] = x[n-2] = \{0, 0, 1, 2, 3, 1\}, \qquad n = 0, \ldots, 5.$$

Figure placeholder

`p1_shifted_impulse.png`

Figure 4: Hand calculation and sketch of $x[n] * \delta[n-2]$.

Since $\delta[n-2]$ is nonzero only at $n = 2$, the convolution sum $\sum_k x[k]\delta[n-k-2]$ keeps only the term $k = n - 2$, giving $x[n-2]$, which is $x[n]$ delayed by two samples.

Part 2: Continuous-Time Convolution

Task 2.1: Step Response of the First Stage

$$h(t) = \frac{1}{\tau} e^{-t/\tau} u(t), \qquad \tau = R_1 C_1 = 0.01 \text{ s}.$$

Hand derivation and final response

Figure placeholder

p2_step_derivation.png

Figure 5: Hand derivation of the first-stage step response $u(t) * h(t)$.

Numerical response

```
# task 2.1 code
```

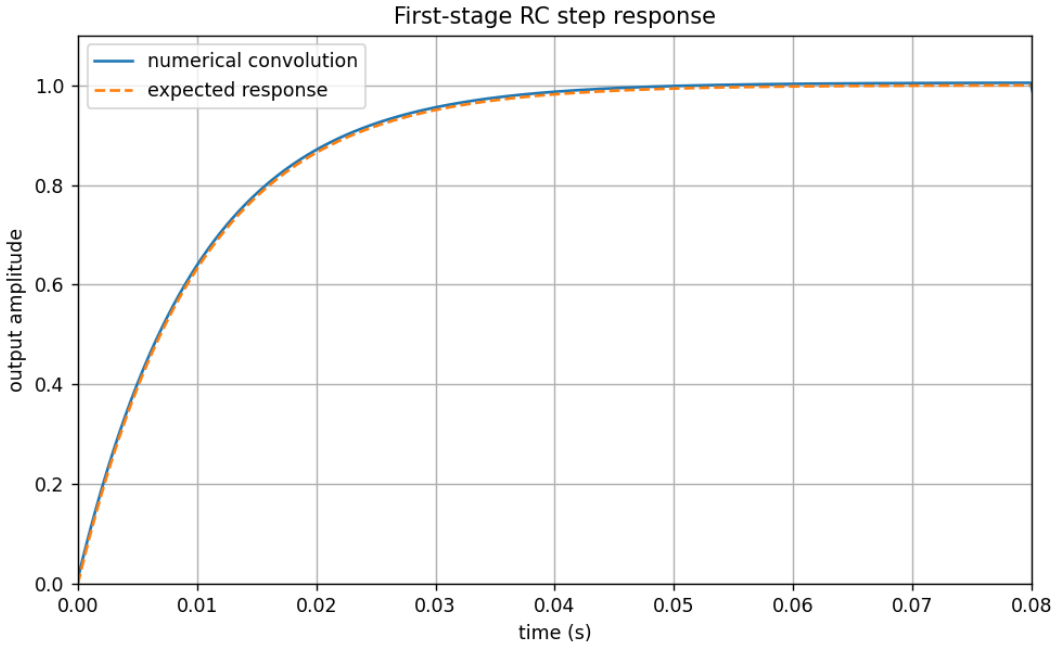


Figure 6: Numerical step response of the first RC stage.

Comparison with the hand calculation

[write here]

Filtering behavior

[write here]

Task 2.2: Cosine Response of the First Stage

$$x(t) = \cos(\omega_0 t)u(t), \qquad \tau = 0.01 \text{ s}.$$

Hand derivation

Figure placeholder

p2_cosine_derivation.png

Figure 7: Hand derivation of the first-stage response to a cosine input.

The required form of the response is

$$y(t) = \frac{\cos\big(\omega_0 t - \tan^{-1}(\omega_0\tau)\big)}{\sqrt{1+(\omega_0\tau)^2}} - \frac{e^{-t/\tau}}{1+(\omega_0\tau)^2}, \qquad t \geq 0.$$

Steady-state response

[write here]

Amplitude and phase expressions

[write here]

$\omega_0\tau$	Amplitude	Phase (°)
0.1	_____	_____
1	_____	_____
10	_____	_____

Table 1: Amplitude and phase of the first-stage output cosine.

Effect of frequency

[write here]

Task 2.3: The Two-Stage Cascade

The two-stage circuit has

$$\tau_1 = 0.01 \text{ s}, \qquad \tau_2 = 0.0047 \text{ s}, \qquad G = 5.7, \qquad \omega = 2\pi f.$$

Cascade derivation

[write here]

The resulting magnitude and phase are

$$M(\omega) = \frac{G}{\sqrt{1+(\omega\tau_1)^2}\sqrt{1+(\omega\tau_2)^2}},$$
$$\phi(\omega) = -\tan^{-1}(\omega\tau_1) - \tan^{-1}(\omega\tau_2).$$

Experimental and analytical magnitudes

The experimental values were taken from the second-stage output measurements in Task 3.4 of Lab 2. The input was 1 V peak-to-peak, so the experimental magnitude was calculated as

$$M_{\text{exp}} = \frac{V_{\text{out,pp}}}{V_{\text{in,pp}}} = \frac{V_{\text{out,pp}}}{1 \text{ V}}.$$

The analytical values were calculated using the magnitude expression above.

Frequency (Hz)	Experimental magnitude	Analytical magnitude
5	5.37	5.380
15	3.80	3.793
30	2.01	1.999
60	0.71	0.718
120	0.20	0.203

Table 2: Experimental and analytical voltage gains of the two-stage RC circuit.

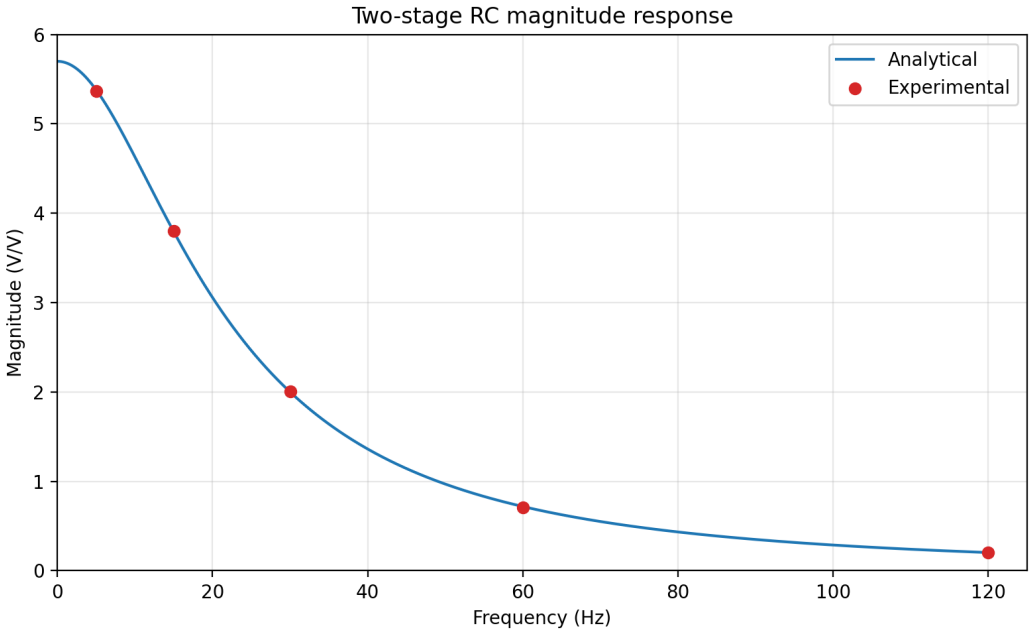


Figure 8: Experimental and analytical magnitude versus frequency for the two-stage RC circuit.

Experimental and analytical agreement

The experimental and analytical values agree closely at all five frequencies. For example, the measured magnitude at 5 Hz was 5.37, compared with an analytical value of 5.380. At 120 Hz, the measured magnitude was 0.20, compared with 0.203 analytically. Small differences could come from component tolerances and measurement uncertainty.

Two-stage filtering behavior

The magnitude decreases as frequency increases. Low-frequency signals pass through with amplification, while higher-frequency signals are strongly reduced. The measured magnitude falls from 5.37 at 5 Hz to 0.20 at 120 Hz.

Both RC stages reduce high-frequency content, so their combined response falls faster than the response of either stage alone. The circuit therefore acts as a two-stage low-pass filter.

Part 3: FIR Audio Filters

Task 3.1: Audio Filtering

The original audio had a sampling rate of $f_s = 16000$ Hz. Each filtered output was normalized before being saved and played.

Python code

```
from scipy.io import wavfile
import numpy as np

fs, clip = wavfile.read("clip.wav")
clip = clip.astype(float)

if clip.ndim > 1:
    clip = clip[:, 0]

clip = clip / np.max(np.abs(clip))

# define the three filters
h_ma = np.ones(64) / 64
h_diff = np.array([1.0, -1.0])

D = int(round(0.2 * fs))
h_echo = np.zeros(D + 1)
h_echo[0] = 1.0
h_echo[D] = 0.6

# filter the original clip
y_ma = np.convolve(clip, h_ma)
y_diff = np.convolve(clip, h_diff)
y_echo = np.convolve(clip, h_echo)

# normalize each output
y_ma = y_ma / np.max(np.abs(y_ma))
y_diff = y_diff / np.max(np.abs(y_diff))
y_echo = y_echo / np.max(np.abs(y_echo))

# save the filtered audio
wavfile.write("low_pass.wav", fs,
              np.int16(y_ma * 32767))
wavfile.write("high_pass.wav", fs,
              np.int16(y_diff * 32767))
wavfile.write("echo.wav", fs,
              np.int16(y_echo * 32767))
```

Moving average filter

The moving average combines 64 samples. Fast changes are averaged out, reducing high-frequency detail and giving the audio a smoother, more muffled sound. It is called a low-pass filter because lower frequencies pass through more strongly than higher frequencies.

Difference filter

The difference filter subtracts the previous sample from the current sample. Slowly changing parts mostly cancel, while rapid changes remain. This reduces low-frequency content and emphasizes sharper changes, so the output is expected to sound thinner and less bass-heavy. It is called a high-pass filter because it emphasizes higher frequencies relative to lower frequencies.

Echo filter

For an echo delay of 0.2 s, the number of samples is

$$D = \text{round}(0.2 f_s) = \text{round}(0.2 \times 16000) = 3200.$$

The first nonzero value of the impulse response passes the original signal. The second produces a copy delayed by 3200 samples and scaled by 0.6. The output therefore contains the original sound followed by a quieter copy 0.2 s later.

Task 3.2: Order Does Not Matter

Python code

The following code uses the original clip and filters defined in Task 3.1.

```
# moving average followed by echo
y1 = np.convolve(np.convolve(clip, h_ma), h_echo)

# echo followed by moving average
y2 = np.convolve(np.convolve(clip, h_echo), h_ma)

# one combined filter
h_combined = np.convolve(h_ma, h_echo)
y3 = np.convolve(clip, h_combined)

# compare the outputs before normalization
print(np.allclose(y1, y2))
print(np.allclose(y1, y3))
```

Numerical agreement and sound

Both comparisons return `True`. The largest difference between the unnormalized outputs is approximately 5.6×10^{-17}, which is just numerical rounding. The saved audio files differ by at most one count in the 16-bit samples, so no audible difference is expected.

These results show that switching the order of the filters or combining them first gives the same output to numerical precision.

Meaning of the combined impulse response

The expression `np.convolve(h_ma, h_echo)` gives the impulse response of one filter that performs both operations. Convolving the clip with this combined filter produces the same result as passing it through the moving average and echo filters separately.

Task 3.3: Moving Average Gain versus Frequency

One-second cosine signals were generated at the eight test frequencies. Each signal had an input amplitude of 1 and was filtered using the 64-sample moving average.

The output amplitude was measured using the maximum absolute value in the second half of the full convolution output, following the lab procedure.

Python code

```
import numpy as np
import matplotlib.pyplot as plt

fs = 16000
freq = [50, 100, 200, 500, 1000, 2000, 4000, 8000]
t = np.arange(0, 1.0, 1.0/fs)
h_ma = np.ones(64) / 64

steady_state_amp = []

# filter each tone and measure its output amplitude
for f in freq:
    x = np.cos(2*np.pi*f*t)
    y = np.convolve(x, h_ma)
    amplitude = np.max(np.abs(y[len(y)//2:]))
    steady_state_amp.append(amplitude)

amp_low_freq = steady_state_amp[0]

fig, ax = plt.subplots()

ax.plot(freq, steady_state_amp, marker="o",
        label="moving average output")
ax.set_xscale("log")
ax.set_xlabel("Frequency (Hz)")
ax.set_ylabel("Output amplitude")
ax.set_title("64-sample moving average response")

ax.axhline(amp_low_freq, color="tab:red",
           linestyle="--",
           label="low frequency amplitude")
ax.axhline(amp_low_freq/2, color="tab:green",
           linestyle="--",
           label="half of low frequency amplitude")

ax.grid(True, alpha=0.3)
ax.legend()
fig.tight_layout()
fig.savefig("task3.3_amplitude_vs_freq.png", dpi=200)
plt.show()
```

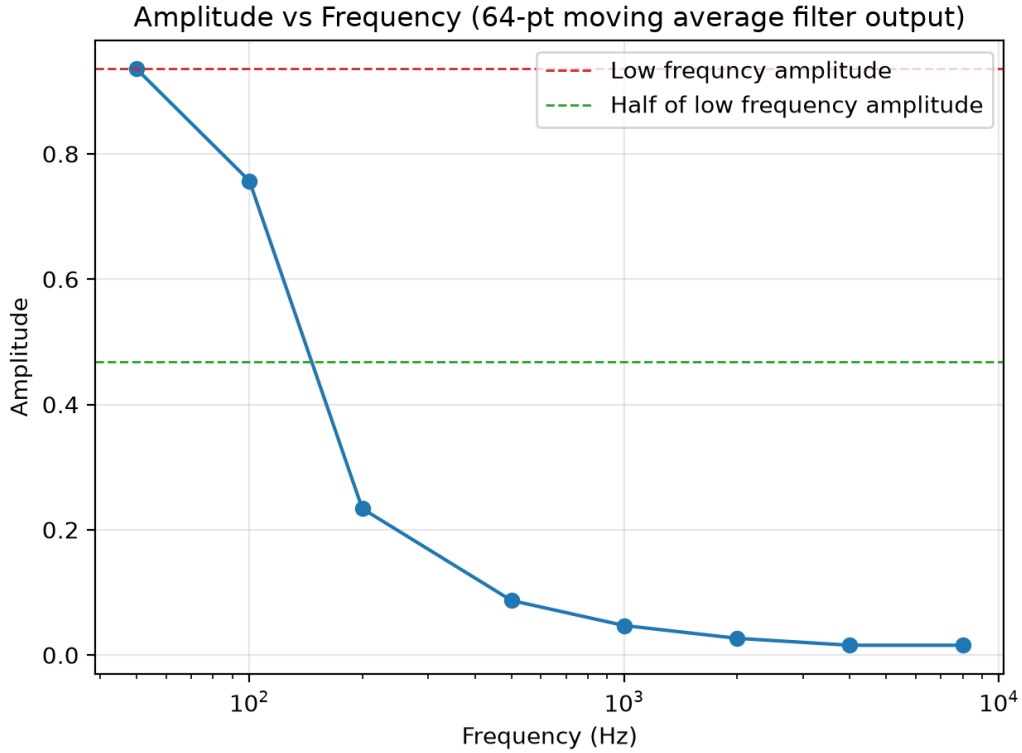


Figure 9: Moving average output amplitude versus frequency on a logarithmic frequency axis.

Half-amplitude frequency

The output amplitude at the lowest test frequency, 50 Hz, was approximately 0.935. Half of this value is

$$\frac{0.935}{2} \approx 0.468.$$

The measured amplitude was approximately 0.757 at 100 Hz and 0.234 at 200 Hz. The half-amplitude crossing is therefore between these frequencies, at roughly

$$f_{\text{half}} \approx 150 \text{ Hz}.$$

This is an estimate because only eight frequencies were tested.

Agreement with the audio filtering

The decreasing output amplitude is consistent with the loss of high-frequency detail described in Task 3.1. Reducing these frequencies makes the filtered audio smoother and more muffled.

Low-pass behavior

The output amplitude is larger at low frequencies and much smaller at high frequencies. This shows that the moving average acts as a low-pass filter.

The small nonzero values at some higher frequencies also include the ending transient, since the specified amplitude measurement includes the tail of the full convolution.

Part 4: IIR Filtering

The IIR filter computes each output sample using the previous output and the current input:

$$y[n] = a\,y[n-1] + b\,x[n],$$

where

$$a = \frac{\tau}{\tau + dt}, \qquad b = \frac{dt}{\tau + dt}, \qquad dt = \frac{1}{f_s}.$$

Task 4.1: IIR Streaming

The step response was calculated using

$$\tau = 0.01 \text{ s}, \qquad f_s = 16000 \text{ Hz}, \qquad dt = 0.0000625 \text{ s}.$$

These values give

$$a \approx 0.993789, \qquad b \approx 0.006211.$$

Python code

```
import numpy as np
import matplotlib.pyplot as plt

tau = 0.01
fs = 16000

dt = 1.0/fs
a = tau/(tau + dt)
b = dt/(tau + dt)

x = np.ones(fs)
y = np.zeros_like(x)

# calculate the output with zero initial conditions
for n in range(len(x)):
    if n == 0:
        y[n] = b*x[n]
    else:
        y[n] = a*y[n-1] + b*x[n]

n = np.arange(len(y))
s_analytic = 1 - np.exp(-n*dt/tau)

fig, ax = plt.subplots()
ax.plot(n, y, label="numeric")
ax.plot(n, s_analytic, "--", label="analytic")

ax.set_title("Analytic vs Numeric RC Step Response")
ax.set_xlabel("Sample (n)")
ax.set_ylabel("Output")
ax.grid(True, alpha=0.3)
ax.legend()

fig.tight_layout()
fig.savefig("task4.1_step_response.png", dpi=200)
plt.show()
```

Figure placeholder

`task4.1_step_response.png`

Figure 10: Numerical IIR step response compared with the analytical RC step response.

Comparison of the step responses

The numerical and analytical curves nearly overlap. Both rise quickly at first and then approach a final value of 1. The small difference comes from approximating the continuous-time system using a finite sampling interval. This shows that the IIR filter closely approximates the RC circuit's step response.

Task 4.2: Audio Filtering and FIR/IIR Comparison

The audio filter used

$$R_1 = 3.3\ \text{k}\Omega, \qquad C_1 = 470\ \text{nF}, \qquad \tau \approx 1.55\ \text{ms}.$$

At $f_s = 16000$ Hz, the coefficients are

$$a = \frac{0.00155}{0.00155 + 0.0000625} \approx 0.961240,$$

$$b = \frac{0.0000625}{0.00155 + 0.0000625} \approx 0.038760.$$

Audio filtering code

```
from scipy.io import wavfile
import numpy as np

fs, clip = wavfile.read("clip.wav")
clip = clip.astype(float)

if clip.ndim > 1:
    clip = clip[:, 0]

clip = clip / np.max(np.abs(clip))

tau = 0.00155
dt = 1.0/fs
a = tau/(tau + dt)
b = dt/(tau + dt)

y = np.zeros_like(clip)

# filter the clip using the previous output sample
for n in range(len(clip)):
    if n == 0:
        y[n] = b*clip[n]
    else:
        y[n] = a*y[n-1] + b*clip[n]

# normalize and save the filtered audio
y_normalized = y / np.max(np.abs(y))
y_i16 = np.int16(y_normalized * 32767)

wavfile.write("clip_RC_filter.wav", fs, y_i16)
```

Sound compared with the FIR moving average

The IIR filter and the FIR moving average both reduce high-frequency content, so a similar smoother and more muffled sound is expected. Their outputs do not need to sound exactly the same because the two filters attenuate frequencies by different amounts.

Computational advantages and disadvantages

The IIR update requires two multiplications and one addition per output sample. It only needs to keep the previous output, making it useful for real-time filtering.

Direct convolution with the 64-sample FIR moving average requires 64 multiplications and 63 additions per output sample. It also needs a history of input samples. This requires more computation and memory than the first-order IIR filter. A running-sum implementation could make this particular moving average more efficient.

Filtering advantages and disadvantages

The IIR filter gives an RC-like low-pass response using only two coefficients. However, feedback means its coefficients must be chosen carefully to keep it stable. Here, $|a| < 1$, so the filter is stable.

An FIR filter with finite coefficients is stable because it has no feedback. Symmetric FIR filters, including this moving average, can provide linear phase. The IIR filter has frequency-dependent phase delay. A disadvantage of FIR filtering is that sharper frequency selection can require a longer filter and more computation.

Task 4.3: IIR Gain versus Frequency

The analytical RC magnitude response uses the time constant from Task 4.2:

$$M(\omega_0) = \frac{1}{\sqrt{1 + (\omega_0 \tau)^2}}, \qquad \omega_0 = 2\pi f, \qquad \tau = 0.00155 \text{ s}.$$

Python code

```
# task 4.3 code
fs = 16000

tau = 0.00155

dt = 1.0/fs
a = tau/(tau+dt)
b = dt/(tau+dt)


freq = [50, 100, 200, 500, 1000, 2000, 4000, 8000]

n = np.arange(0, 1.0, dt)

h_ma = np.ones(64)/64 #length-64 moving average filter


#create list of one second pure tones at each test frequency
tones = []
for f in freq:
    tones.append(np.cos(2*np.pi*f*n))

y_iir = []
for t in tones:
    out = np.zeros(len(t))
    for k in range(len(t)):  # k is the current sample of the live stream
        if k == 0:
            out[k] = b*t[k]
        else:
            out[k] = a*out[k-1] + b*t[k]
    y_iir.append(out)

y_fir = []
for t in tones:
    y_fir.append(np.convolve(t, h_ma))

#y_analytic
f_dense = np.logspace(np.log10(freq[0]), np.log10(freq[-1]), 500)
H_analytic = 1/np.sqrt(1 + (2*np.pi*f_dense*tau)**2)

#measure steady state amplitude of output at each frequency
steady_state_amp_iir = []
for out in y_iir:
    steady_state_amp_iir.append(np.max(np.abs(out[len(out)//2:])))

steady_state_amp_fir = []
for out in y_fir:
    steady_state_amp_fir.append(np.max(np.abs(out[len(out)//2:])))
```

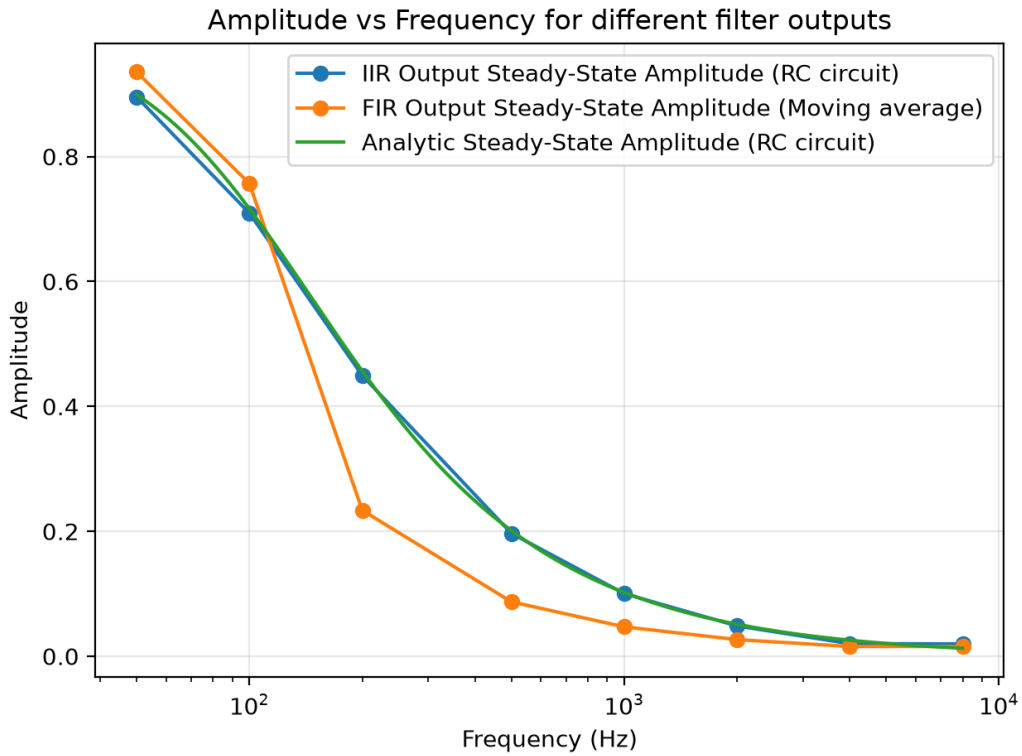


Figure 11: Output amplitude versus frequency for the FIR moving average, the IIR filter, and the analytical RC response.

Agreement between the IIR and analytical responses

[write here]

Comparison of the three curves and low-pass behavior

[write here]

Conclusion

Summary and what was learned

[write here]

Take-away messages

- *[write here]*
- *[write here]*
- *[write here]*